



**The Saigon International
University**

KHOA KỸ THUẬT & KHOA HỌC MÁY TÍNH

**TIỂU LUẬN CUỐI KỲ
MÔN KIẾN TRÚC MÁY TÍNH**

ĐỀ TÀI:

**CÁCH TIẾP CẬN CHỊU LỖI CHỦ ĐỘNG CHO ĐIỆN TOÁN ĐÁM
MÂY DỰA TRÊN HỆ MỜ TAKAGI-SUGENO VÀ THUẬT TOÁN
SIMULATED ANNEALING**

SINH VIÊN: NÔNG VĂN TÌNH

MSSV: 8480101250028

LỚP: 25MCS2

GIẢNG VIÊN: GS.TS. TRẦN CÔNG HÙNG

HK2 - NĂM HỌC 2025-2026

CÁCH TIẾP CẬN CHỊU LỖI CHỦ ĐỘNG CHO ĐIỆN TOÁN Đám Mây DỰA TRÊN HỆ THỐNG MỜ TAKAGI-SUGENO VÀ THUẬT TOÁN SIMULATED ANNEALING

A PROACTIVE FAULT TOLERANCE APPROACH FOR CLOUD COMPUTING BASED ON TAKAGI-SUGENO FUZZY SYSTEM AND SIMULATED ANNEALING ALGORITHM

International Journal of Latest Trends in Engineering and Technology Vol.(13) Issue(3), pp.016–027 DOI: <http://dx.doi.org/10.21172/1.133.03> e-ISSN: 2278-621X

Tác giả: Khiet Bui Thanh¹, Linh Phung Dieu², Sam Dang Thi Hong³, Tran Vu Pham⁴, Hung Tran Cong⁵

¹ Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa TP. Hồ Chí Minh, ĐHQG-HCM, Việt Nam ² Cục Công nghệ thông tin, Bộ Công an, Việt Nam ³ Khoa Kỹ thuật – Công nghệ, Trường Đại học Thủ Dầu Một, Bình Dương, Việt Nam ⁴ Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa TP. Hồ Chí Minh, ĐHQG-HCM, Việt Nam ⁵ Khoa Đào tạo & Khoa học Công nghệ, Học viện Công nghệ Bưu chính Viễn thông, TP. Hồ Chí Minh, Việt Nam

Tóm tắt

Khả năng chịu lỗi (Fault Tolerance) trong điện toán đám mây (Cloud Computing) là một thách thức lớn. Các chính sách chịu lỗi trong điện toán đám mây có thể được chia thành hai loại: thụ động (passive) và chủ động (proactive). Trong bài báo này, chúng tôi đề xuất một cách tiếp cận chịu lỗi chủ động cho điện toán đám mây theo mô hình IaaS. Cách tiếp cận này gồm hai thành phần chính: dự đoán lỗi (fault prediction) và phòng ngừa lỗi (fault prevention). Lỗi trên các máy vật lý (Physical Machine – PM) được dự đoán bằng hệ thống mờ Takagi-Sugeno (Takagi-Sugeno Fuzzy System), vốn dựa trên cách tiếp cận theo tri thức (knowledge-based). Chiến lược phòng ngừa lỗi dựa trên cân bằng Nash (Nash Equilibrium) của mô hình trò chơi di chuyển máy ảo (VM migration game). Chúng tôi đề xuất thuật toán PFTSA để giải quyết bài toán di chuyển dựa trên thuật toán Simulated Annealing (SA), đáp ứng nhu cầu về khả năng mở rộng (scalability) của điện toán đám mây. Thuật toán PFTSA có thể tìm được kết quả tối ưu hoặc gần tối ưu trong khoảng thời gian chấp nhận được nhờ dựa vào việc ngẫu nhiên hóa cách xác định các chiến lược di chuyển, qua đó tăng khả năng khám phá không gian lời giải. Trong phần đánh giá, chúng tôi trình bày ảnh hưởng của việc điều chỉnh các tham số ϵ và cooling rate (tốc độ làm nguội) đến hiệu quả của thuật toán PFTSA. Sau đó, kết quả của thuật toán PFTSA được so sánh (benchmark) với thuật toán PSOVM ở các khía cạnh: mức độ cân bằng tải, tài nguyên bị lãng phí, thời gian thực thi và giá trị hàm mục tiêu.

Từ khóa

Khả năng chịu lỗi (Fault tolerance), Điện toán đám mây IaaS (IaaS Cloud Computing), Hệ thống mờ Takagi-Sugeno (Takagi-Sugeno Fuzzy System), Thuật toán Simulated Annealing.

1. GIỚI THIỆU

Điện toán đám mây ngày càng trở nên phổ biến nhờ những lợi ích mà nó mang lại cho khách hàng, chẳng hạn như triển khai ứng dụng một cách linh hoạt, đơn giản hóa việc thêm và giải phóng tài nguyên khi chạy ứng dụng, trong khi chi phí tài nguyên được tính dựa trên mức sử dụng trong từng lần dùng. Thay vì sử dụng một hoặc nhiều máy chủ vật lý, người dùng có thể sử dụng tài nguyên ảo hóa thông qua môi trường Internet dưới dạng Cơ sở hạ tầng như một dịch vụ (Infrastructure as a Service – IaaS) hoặc Nền tảng như một dịch vụ (Platform as a Service – PaaS), bao gồm cả các API để phát triển ứng dụng trên một nền tảng công nghệ cụ thể. Ngoài ra, người dùng có thể sử dụng Phần mềm như một dịch vụ (Software as a Service – SaaS), vốn chủ yếu được cung cấp dưới dạng ứng dụng nền web và truy cập từ xa. Ở cấp độ người dùng, một hệ thống máy tính đạt được độ tin cậy khi nó thực hiện đúng các tác vụ được giao và cho ra một số kết quả đáng tin cậy.

Một trong những vấn đề then chốt cần cân nhắc khi xây dựng điện toán đám mây là khả năng sẵn sàng (availability) của dịch vụ. Vì vậy, trong hầu hết các trường hợp luôn cần có các biện pháp phòng ngừa. Một dịch vụ điện toán đám mây phải có khả năng nhận diện và ứng xử phù hợp để bảo đảm sự trơn tru cũng như chất lượng dịch vụ (Quality of Service – QoS). Lỗi (Fault) là một trạng thái bất thường ở một hoặc nhiều bộ phận của hệ thống. Một lỗi có thể là nguyên nhân, hoặc cũng có thể không phải là nguyên nhân, khiến hệ thống hoạt động không đúng với thiết kế ban đầu [1]. Ngay khi lỗi vừa xảy ra, hệ thống vẫn thực hiện các biện pháp phục hồi dữ liệu để tránh mất mát dữ liệu. Khả năng này được định nghĩa là khả năng chịu lỗi (fault tolerance) [2–4]. Hệ thống bị coi là thất bại nếu quá trình hoạt động thực tế không đáp ứng tốt so với thiết kế ban đầu. Theo đó, một trục trặc (malfunction) là hệ quả của một lỗi xảy ra ở các thành phần của hệ thống. Hệ thống được xem là chịu được lỗi nếu nó vẫn hoạt động ở mức chấp nhận được theo thiết kế ban đầu ngay cả khi có lỗi xuất hiện. Nhiều ứng dụng trong các ngành khác nhau đòi hỏi một hệ thống có khả năng duy trì chức năng ngay cả khi xảy ra lỗi, chẳng hạn như hệ thống ngân hàng, hệ thống điều khiển, hệ thống vận hành.

Các chính sách chịu lỗi trong điện toán đám mây [2] có thể được chia thành hai nhóm:

Chính sách chịu lỗi thụ động (passive fault tolerance) nhằm giảm thiểu hậu quả của các lỗi phát sinh do quá trình thực thi ứng dụng hoặc do di chuyển dịch vụ. Có một số kỹ thuật cho chính sách này như checkpoint/restart [5–7], nhân bản (replication) [8–10], gửi lại tác vụ (task resubmission), cấu hình do người dùng định nghĩa. Kỹ thuật Checkpoint/Restart cho phép hệ thống khởi động lại ở trạng thái gần với trạng thái checkpoint khi một tác vụ gặp lỗi. Nhân bản (Replication) là một kỹ thuật phổ biến trong phòng ngừa lỗi. Nhân bản là quá trình duy trì nhiều bản sao của một hệ thống hoặc một đối tượng. Trong kỹ thuật này, các yêu cầu từ phía máy khách được chuyển tới một trong các bản sao đã được tạo ra. Hiện nay có nhiều công cụ cho phép triển khai kỹ thuật này: HA Proxy, Hadoop và AmazonEC2 [11]. Gửi lại tác vụ (Task Resubmission) được sử dụng rộng rãi trong các hệ thống luồng công việc (workflow) phục vụ nghiên cứu khoa học. Mỗi khi phát hiện lỗi, tác vụ sẽ được khởi động lại và thực thi trên chính tài nguyên hiện tại hoặc trên các tài nguyên khác [12, 13]. Xử lý lỗi do người dùng định nghĩa (User-Defined Error Handling) là việc người dùng cung cấp cách xử lý cho một số trường hợp lỗi của tác vụ trong một luồng công việc cụ thể. Kiểu xử lý này cố gắng che giấu lỗi trong quá trình thực thi tác vụ của luồng công việc. Trong một luồng công việc, lỗi có thể xảy ra ở các cấp độ sau: cấp phần cứng và cấp hệ điều hành. Cấp phần cứng là cấp thấp nhất, có thể là sự cố của máy tính hoặc mạng. Ở cấp hệ điều hành có thể xảy ra tràn bộ nhớ, tràn không gian đĩa cứng, hoặc vượt quá thời gian chờ (timeout) của CPU. Lỗi cũng có thể xảy ra do tắc nghẽn mạng, v.v.

Chính sách chịu lỗi chủ động (proactive fault tolerance) giúp ứng dụng hoặc dịch vụ của bạn luôn thực thi đúng đắn bằng cách tránh các lỗi tiềm ẩn thông qua các biện pháp phòng ngừa. Hai kỹ thuật của chính sách này là Di chuyển chủ động trước lỗi (Preemptive Migration) và Làm mới phần mềm (Software Rejuvenation). Ứng dụng được giám sát và phân tích thông qua một hệ thống giám sát. Dựa trên phản hồi đó, hệ thống sẽ dự đoán liệu một lỗi có sắp xảy ra hay không; nếu có, hệ thống sẽ di chuyển ứng dụng đến một nơi an toàn hơn. Đó là di chuyển chủ động trước lỗi (preemptive migration) [14]. Làm mới phần mềm (Software Rejuvenation) là kỹ thuật được thiết kế để khởi động lại hệ thống theo định kỳ. Khi khởi động lại, hệ thống sẽ xóa trạng thái trước đó và tại thời điểm này, lỗi hệ thống cũng không còn tồn tại [15].

Với mục tiêu xây dựng thành công khả năng chịu lỗi cho hệ thống, hệ thống cần có các thành phần phát hiện lỗi (fault detection), chẩn đoán lỗi (fault diagnosis) và phòng ngừa lỗi (fault prevention). Phát hiện lỗi và chẩn đoán lỗi dựa trên việc đo đạc, quan sát các biến số và trạng thái của hệ thống. Dữ liệu được giám sát cần được phân tích và đánh giá dựa trên tri thức của người vận hành. Phương pháp phát hiện và chẩn đoán lỗi có thể được xem xét trong khuôn khổ cách tiếp cận theo tri thức (knowledge-based approach) [16]. Các phương pháp phát hiện thay đổi (change detection) và phân loại (classification) thường được dùng để so sánh giá trị đo được của các thuộc tính trong hệ thống với giá trị đo được trong quy trình chuẩn. Kết quả của các thay đổi này đến từ tín hiệu đo được, mô hình tín hiệu hoặc mô hình quy trình, vốn được xem là các triệu chứng phân tích (analytic symptoms). Ngoài ra, các triệu chứng heuristic có thể được tạo ra bằng cách sử dụng thông tin định tính từ người vận hành. Nhiệm vụ của chẩn đoán lỗi là xác định hình thức, quy mô, vị trí lỗi có khả năng xảy ra, và khoảng thời gian cần thiết để phát hiện chúng. Việc xác định lỗi từ các mẫu triệu chứng có thể áp dụng các phương pháp phân loại. Tuy nhiên, thông tin phong phú hơn về quan hệ lỗi–triệu chứng, chẳng hạn như cây lỗi–triệu chứng (fault-symptom trees) hoặc luật if-then, có thể được khám phá bằng các phương pháp suy luận (inference). Dữ liệu thống kê (thời gian trung bình đến khi xảy ra lỗi, xác suất lỗi) có thể được tính toán dựa trên kinh nghiệm. Theo đó, những kinh nghiệm như vậy có thể được biểu diễn dưới dạng các biến ngôn ngữ (linguistic variables) như nhỏ, trung bình, lớn, hoặc các số mờ (vague numbers) kiểu “gần bằng” một giá trị nào đó. Chúng có thể được biểu diễn dưới dạng các số tin cậy (confidence numbers), các hàm thành viên (membership

functions) của tập mờ, hoặc các hàm mật độ xác suất thông qua các đánh giá thống kê tại một thời điểm nào đó. Trong nghiên cứu này, chúng tôi đề xuất một cách tiếp cận chịu lỗi chủ động cho các ứng dụng đa tầng (multi-tier) trong điện toán đám mây IaaS. Lỗi trên máy vật lý được dự đoán bằng hệ thống mờ Takagi-Sugeno, vốn dựa trên tri thức của các chuyên gia. Chiến lược phòng ngừa lỗi dựa trên cân bằng Nash của mô hình trò chơi di chuyển máy ảo. Chúng tôi đề xuất thuật toán PFTSA để giải quyết bài toán này dựa trên thuật toán Simulated Annealing (SA), một thuật toán metaheuristic, nhằm đáp ứng nhu cầu về khả năng mở rộng của điện toán đám mây.

Các phần còn lại được trình bày như sau. Phần II là các nghiên cứu liên quan. Kiến trúc hệ thống, mô hình dự đoán lỗi PM và mô hình trò chơi di chuyển máy ảo được trình bày trong Phần III. Phần IV giới thiệu thuật toán SA và đề xuất thuật toán PFTSA cho việc di chuyển máy ảo. Phần V là thí nghiệm để đánh giá phương pháp đề xuất. Cuối cùng, kết luận và hướng phát triển tiếp theo được trình bày trong Phần VI.

2. CÁC NGHIÊN CỨU LIÊN QUAN

Có rất nhiều loại lỗi trong điện toán đám mây [17–23]. Jhavar và Piuri [20] phân loại lỗi thành hai loại, gồm lỗi sập (crash faults) và lỗi Byzantine (byzantine faults), được mô tả như sau. Lỗi sập xảy ra khi một hoặc nhiều thành phần bị hỏng, chẳng hạn như nguồn điện, đĩa lưu trữ, chip nhớ, bộ xử lý, thiết bị chuyển mạch và định tuyến mạng, v.v. Sự sai lệch giữa kết quả kỳ vọng ban đầu và kết quả thực tế dẫn đến các điều kiện không thể dự đoán được, gọi là lỗi Byzantine. Sự xuất hiện của lỗi Byzantine ảnh hưởng đến cấu trúc logic của hệ thống. Việc kiểm thử và kết luận chính xác về các điều kiện Byzantine có thể khá khó khăn do tính phức tạp và đa dạng tồn tại trong hệ thống. Tuy nhiên, có nhiều loại lỗi có thể được phân loại đồng thời là cả lỗi sập và lỗi Byzantine.

Yuqiang Luo và cộng sự [19] đã giới thiệu những kết quả mới cho thấy logic mờ (fuzzy logic) là một phương pháp hiệu quả để phân tích và tổng hợp thông tin mạng. Ngoài ra, logic mờ còn hỗ trợ các luật điều khiển phi tuyến cho các hệ thống phức tạp. Có một số hướng nghiên cứu mới như: Hệ thống mạng hai chiều (2D Networked Systems), Hệ thống điều khiển mạng không dây (Wireless Networked Control Systems – WNCSSs), Chất lượng dịch vụ (QoS) của các hệ thống mạng, Điều khiển truy cập mờ trong các hệ thống mạng mở, Bộ điều khiển thông minh trong các hệ thống mạng. Logic mờ sẽ được nghiên cứu trong lĩnh vực lọc (filtering) và phát hiện lỗi cho mạng.

Daniel J. Dean [21] đề xuất PerfCompass, một công cụ định vị lỗi bất thường về hiệu năng ở thời điểm chạy (runtime performance anomaly fault localization). PerfCompass sử dụng một hệ thống theo dõi (tracing) ở cấp nhân (kernel-level) nhẹ và thực thi trên một hệ thống phân tích dấu vết trực tuyến (online trace analysis system) để trích xuất các thuộc tính lỗi từ khối lượng lớn dữ liệu hệ thống thô trong lúc chạy. PerfCompass chẩn đoán chính xác tất cả các lỗi được kiểm thử. Bài báo sử dụng kỹ thuật phân tích dấu vết trực tuyến, hướng tới một công cụ phát hiện lỗi tích hợp sẵn trong IaaS. PerfCompass nhẹ và không xâm lấn đối với IaaS, không yêu cầu bất kỳ mã nguồn hay công cụ đo đặc bổ sung ở thời điểm chạy nào.

Chris Schneider [22] so sánh hiệu quả của một số loại nguyên hàm ngẫu nhiên (stochastic primitives) khác nhau bằng cách sử dụng học không giám sát (unsupervised learning) để xác định theo hướng heuristic các nguyên nhân gốc rễ của lỗi. Kết quả cho thấy UBL và ADF có thể xác định chính xác cả sự hiện diện lẫn nguyên nhân tiềm ẩn của lỗi trong khoảng thời gian ngắn bằng cách sử dụng dữ liệu chưa gán nhãn. Các biến số liên quan và mối quan hệ giữa chúng có thể được nghiên cứu sâu hơn. Bài báo này sử dụng học không giám sát để tìm nguyên nhân của lỗi. Cụ thể, bài báo gợi ý rằng các nghiên cứu tiếp theo nên xem xét Mạng ngẫu nhiên sinh (Generative Stochastic Networks) thay cho RBM, so sánh giữa phần mềm tự chữa lành (self-healing software) và con người, cùng các kỹ thuật tiến hóa.

Claude Delpha [23] đề xuất sử dụng một mô hình giấu dữ liệu (data-hiding model) để mô tả thông tin ẩn được nhúng trong tín hiệu của máy chủ nhằm giám sát. Sau đó, mô hình này sẽ phát hiện và mô tả đặc điểm của thông tin ẩn để từ đó phát hiện lỗi. Trước đây đã có cách tiếp cận dựa trên mô hình (model-based) và cách tiếp cận dựa trên tri thức (knowledge-based). Các tác giả đề xuất một cách tiếp cận dựa trên thông tin ẩn (hidden information-based). Có các quá trình giấu thông tin và khôi phục ngược thông tin. Các tác giả đưa ra một mô hình mới về phát hiện và chẩn đoán lỗi bằng cách liên hệ tương đồng với mô hình giấu tin (information hiding paradigm). Bài báo này mở ra nhiều hướng nghiên cứu trong tách nguồn (source separation), các kỹ thuật tối ưu hóa, thống kê, và lý thuyết trò chơi không hợp tác (non-cooperative game theory).

Một cơ chế chịu lỗi chủ động có thể xử lý các gián đoạn có thể xảy ra trước khi lỗi thực sự xuất hiện. Các trạng thái hệ

thống được giám sát liên tục và một lỗi xảy ra sẽ được ước lượng dựa trên nguyên lý của các phương pháp này, vốn dựa trên kinh nghiệm và kỳ vọng. Khả năng chịu lỗi mạnh mẽ là yêu cầu bắt buộc ở tầng hạ tầng để cung cấp dịch vụ. Lỗi xảy ra ở tầng vật lý có thể gây ra những hậu quả nghiêm trọng nhất. Các cơ chế phát hiện lỗi tương ứng với từng lớp dịch vụ (IaaS/PaaS/SaaS) đã được áp dụng trong nghiên cứu. Ví dụ, lỗi ở tầng IaaS thường được phát hiện thông qua các kỹ thuật giám sát trạng thái, chẳng hạn như giao thức heartbeat [24]. Cách tiếp cận chịu lỗi kiểu tự chữa lành (self-healing) [25, 26] được định nghĩa là khả năng của một hệ thống tự động phục hồi khỏi lỗi bằng cách áp dụng định kỳ một quy trình phục hồi cụ thể. Cách tiếp cận này đòi hỏi phải xem xét cẩn thận nhiều khía cạnh của lỗi (vị trí, thời gian, cường độ, v.v.) để hành động thành công. Trong cách tiếp cận di chuyển chủ động trước lỗi (pre-emptive migration), chỉ báo tiền lỗi (pre-fault indicator) trên hệ thống được dùng để dự đoán bất kỳ lỗi nào có thể xảy ra ở các nút trong tương lai gần. Các tác vụ đang được xử lý tại các nút lỗi sẽ được chuyển tiếp sang các nút an toàn khác [27].

3. KIẾN TRÚC HỆ THỐNG

3.1. Kiến trúc hệ thống

Trong phần này, mô hình dịch vụ hạ tầng (infrastructure service model) của điện toán đám mây được trình bày. Cụ thể, tài nguyên ảo hóa được nhà cung cấp đám mây IaaS cung cấp dưới dạng các máy ảo (VM). Các máy ảo có CPU, bộ nhớ, dung lượng lưu trữ, mạng và khả năng sẵn sàng ở những mức độ khác nhau. Một ứng dụng trên đám mây có thể được triển khai trên một cụm máy ảo (VMs cluster) vốn có thể không đồng nhất (heterogeneous). Các máy ảo được phân loại theo các vai trò xử lý cụ thể như xử lý tính toán, bộ nhớ, lưu trữ, GPU, v.v. Những khách hàng sở hữu các ứng dụng đa tầng (multitier) sẽ lựa chọn các máy ảo phù hợp cho ứng dụng của mình, chẳng hạn như máy ảo tối ưu cho tính toán dành cho ứng dụng web và máy ảo tối ưu cho bộ nhớ dành cho các ứng dụng xử lý cơ sở dữ liệu hiệu năng cao, v.v. Để bảo đảm QoS cho khách hàng, các nhà cung cấp đám mây thường đặt ra một số mục tiêu QoS nhất định, chẳng hạn như thời gian phản hồi, thông lượng (throughput), độ trễ (latency), thời gian thực thi và thời gian giao dịch. Việc kiểm soát chất lượng sẽ tạo ra những dịch vụ có uy tín, mang lại sự hài lòng cho khách hàng và qua đó tăng số lượng người dùng cũng như doanh thu.

Để triển khai việc tự động điều chỉnh tài nguyên trong hệ thống điện toán đám mây, chúng tôi sử dụng vòng lặp MAPE-K để thực hiện các tác vụ giám sát (Monitor), phân tích (Analyze), lập kế hoạch (Plan) và thực thi (Execute). Mô-đun giám sát là một dịch vụ thường được các nhà cung cấp dịch vụ đám mây cung cấp, chẳng hạn như Amazon CloudWatch và Google Cloud Monitoring, giúp giám sát liên tục hiệu năng của các ứng dụng đang chạy. Thông tin nhận được từ mô-đun giám sát được sử dụng trong giai đoạn phân tích và lập kế hoạch để ước lượng nhu cầu tài nguyên trong tương lai và lập kế hoạch cho một hành động điều chỉnh tài nguyên phù hợp. Các tham số đầu vào có thể là tải công việc (job load), các phép đo QoS và các ngưỡng (thresholds). Trong mỗi cụm, máy ảo chạy một ứng dụng đa tầng có một Broker chịu trách nhiệm phân phối các yêu cầu tới các máy ảo khác nhau. Các cụm máy ảo có thể không đồng nhất, vì vậy chúng tôi sử dụng các chiến lược cân bằng tải theo trọng số (weighted load balancing) để phân phối yêu cầu tới các máy ảo khác nhau tương ứng với năng lực của chúng. Mỗi máy ảo chạy một tác nhân cục bộ (local agent) để truy vấn các tham số hiệu năng hiện tại, chẳng hạn như tải CPU và bộ nhớ.

Hình 1. Kiến trúc hệ thống

(Sơ đồ: vòng lặp MAPE-K gồm Monitor → Analyze → Plan → Execute, với luồng “Metrics observed” (các chỉ số được quan sát) đi vào và luồng “Create/Destroy/Migrate VMs” (Tạo/Hủy/Di chuyển máy ảo) đi ra. Bên phải là cụm điện toán đám mây gồm các Ứng dụng 1, 2, 3 chứa nhiều VM, đặt trên các tầng Ảo hóa (Virtualization) và Tài nguyên vật lý (Physical Resources).)

Điện toán đám mây cung cấp tài nguyên cho các ứng dụng của khách hàng. Hình 1 minh họa cơ chế Tự giám sát MAPE-K (MAPE-K Self-Monitoring) cho điện toán đám mây IaaS, vốn cung cấp cụm máy ảo để chạy ứng dụng.

3.2. Dự đoán lỗi PM dựa trên hệ thống mờ Takagi-Sugeno

Chúng tôi sử dụng phương pháp suy diễn mờ Takagi-Sugeno (Takagi-Sugeno fuzzy inference), trong đó mỗi miền có một hàm rõ ràng theo các biến đầu vào và cung cấp một phép xấp xỉ phi tuyến bằng các luật tuyến tính. Suy diễn mờ là quá trình ánh xạ một bộ điều khiển đầu vào tới một tập các đầu ra điều khiển thông qua các tập mờ. Tiềm năng của logic mờ nằm ở khả năng ước lượng phi tuyến bằng cách biểu diễn tri thức tương tự như cách con người suy luận. Suy

diễn mờ là quá trình ánh xạ một tập các giá trị điều khiển đầu vào thành các giá trị điều khiển đầu ra thông qua các tập mờ. Thành phần điều khiển mờ thường được áp dụng cho các bài toán khó biểu diễn bằng các mô hình toán học tường minh. Các thành phần của điều khiển mờ bao gồm:

- **Cơ sở tri thức (Knowledge Base):** là các luật được cấu hình động theo các tham số ngưỡng để phù hợp với nhiều điều kiện của hệ thống.
- **Mờ hóa (Fuzzification):** là quá trình đưa tập dữ liệu đầu vào về dạng mờ để phục vụ cho quá trình xấp xỉ. Để mờ hóa hiệu quả, cần phải xử lý dữ liệu trước.
- **Suy luận xấp xỉ (Approximate reasoning):** là quá trình dựa trên các luật do cơ sở tri thức cung cấp, đưa ra kết luận về trạng thái của hệ thống.
- **Giải mờ (Defuzzification):** đưa ra hành động hợp lý cho từng trạng thái cụ thể của ứng dụng.

Hình 2. Hệ thống mờ Takagi-Sugeno

(Sơ đồ luồng: Tiền xử lý (Pre-Processing) → Mờ hóa (Fuzzification) → Suy luận xấp xỉ (Approximate Reasoning) → Giải mờ (Defuzzification); trong đó bước Suy luận xấp xỉ tương tác với Cơ sở tri thức (Knowledge Base) và Chuyên gia (Expert).)

Đơn vị Giám sát tự động (Automated Monitoring Unit) giám sát việc sử dụng tài nguyên. Tài nguyên được sử dụng phải đáp ứng các mục tiêu SLA. Bộ giám sát thu thập dữ liệu về tình trạng sử dụng tài nguyên và chuyển cho bộ điều khiển (controller). Bộ điều khiển đưa ra các lệnh để điều chỉnh tài nguyên một cách phù hợp cho từng yêu cầu công việc; cụ thể, bộ điều khiển sẽ ra quyết định tăng, giảm hoặc di chuyển máy ảo cho các công việc cụ thể.

Dựa trên cơ chế suy diễn của hệ thống Takagi-Sugeno, hiệu ứng đầu ra a với $\mathbf{x} = (x_1, x_2, \dots)$ được xấp xỉ bằng công thức sau:

$$a(x) = \frac{\sum_i \alpha_i(x) a_j^i}{\sum_i \alpha_i(x)} \quad (1)$$

trong đó,

$$\alpha_i(x) = \min(\mu_1^i(x_1), \mu_2^i(x_2), \dots, \mu_N^i(x_N))$$

là giá trị chân trị (truth value) của luật; $y = \dots$ là một hành động được chọn của luật R_i .

(Ghi chú của người dịch: một phần biểu thức của y trong bản gốc bị mờ, chỉ đọc được rằng đây là hành động được chọn ứng với luật R_i .)

Với hệ thống mờ dựa trên các luật mờ, các bước suy luận mờ (các phép suy diễn tương ứng với các luật mờ IF – THEN) được thực hiện bởi các hệ suy diễn mờ như sau:

- So sánh các biến đầu vào với các hàm phụ thuộc trong phần giả thiết để thu được giá trị phụ thuộc (hay giá trị có thể so sánh được) của từng nhân ngôn ngữ. (Bước này thường được gọi là mờ hóa.)
- Sinh ra các kết quả chất lượng (dạng mờ hoặc dạng tập) của mỗi luật tùy theo mức độ kích hoạt (degree of firing).
- Tích hợp các kết quả chất lượng để tạo thành một tập đầu ra (bước này được gọi là giải mờ).

Hình 3. Biểu đồ mờ cho PM - (a) Biểu đồ mờ tải của PM (Fuzzy PM load chart): ba tập mờ Thấp (Low), Trung bình (Medium), Cao (High) trên trục tải, với các điểm mốc l_1, l_2, l_3, l_4 . - **(b) Biểu đồ mờ nhiệt độ của PM (Fuzzy PM temperature chart):** ba tập mờ Thấp (Low), Trung bình (Medium), Cao (High) trên trục nhiệt độ, với các điểm mốc t_1, t_2, t_3 .

Tải của PM được tính:

[Công thức (2) không đọc rõ trong bản PDF — phần thân công thức bị bỏ trống/không hiển thị trong bản gốc.]

Jialei Liu [28] đề xuất:

$$\text{temp}(t | A, \omega, t_i, t_{i+1}) = \begin{cases} A \sin(\dots) \\ \vdots \end{cases} \quad (3)$$

[Công thức (3) không đọc rõ trong bản PDF — vẽ phải của hàm từng khúc bị cắt khỏi lề trang, chỉ đọc được phần đầu “A·sin(...)”.]

Các hàm thành viên (membership functions) cho tải và nhiệt độ của PM:

[Công thức (4) — $\mu_{\text{low}}(\text{load})$: không đọc rõ trong bản PDF; chỉ thấy nhãn vẽ trái, vẽ phải bị cắt khỏi lề trang.]

[Công thức (5) — $\mu_{\text{low}}(\text{temp})$: không đọc rõ trong bản PDF; chỉ thấy nhãn vẽ trái, vẽ phải bị cắt khỏi lề trang.]

[Công thức (6) — $\mu_{\text{medium}}(\text{load})$: không đọc rõ trong bản PDF; chỉ thấy nhãn vẽ trái, vẽ phải bị cắt khỏi lề trang.]

[Công thức (7) — $\mu_{\text{medium}}(\text{temp}) = \dots$: không đọc rõ trong bản PDF; chỉ thấy nhãn vẽ trái, vẽ phải bị cắt khỏi lề trang.]

[Công thức (8) — $\mu_{\text{high}}(\text{load})$: không đọc rõ trong bản PDF; chỉ thấy nhãn vẽ trái, vẽ phải bị cắt khỏi lề trang.]

[Công thức (9) — $\mu_{\text{high}}(\text{temp})$: không đọc rõ trong bản PDF; chỉ thấy nhãn vẽ trái, vẽ phải bị cắt khỏi lề trang.]

(Ghi chú của người dịch: Các công thức (4)–(9) là các hàm thành viên tương ứng với ba mức Thấp/Trung bình/Cao của tải và nhiệt độ trong Hình 3. Trong bản PDF gốc, phần định nghĩa (vẽ phải) của các hàm này bị cắt khỏi vùng in nên không thể đọc chính xác; do đó không được suy đoán, chỉ giữ lại nhãn ký hiệu như trên.)

3.3. Mô hình trò chơi di chuyển máy ảo

Để mô hình hóa bài toán này bằng lý thuyết trò chơi (game theory), chúng tôi coi các ứng dụng đa tầng là những người chơi (players) trong trò chơi di chuyển máy ảo. Giả sử trong hạ tầng đám mây có một số máy vật lý (PM). Nhờ công nghệ ảo hóa, các PM có thể tự triển khai các máy ảo. Hệ thống điện toán đám mây cung cấp tài nguyên máy ảo cho vector các ứng dụng đa tầng $A = \{A_1, A_2, \dots\}$. Một vector di chuyển tài nguyên $\Phi = \{\Phi_1, \Phi_2, \dots\}$ xác định số lượng máy ảo được cấp phát cho mỗi ứng dụng trên tất cả các PM một cách hợp lý. Chiến lược di chuyển máy ảo cho mỗi ứng dụng được biểu diễn bằng một ma trận không âm, với hàng ứng với từng tầng (tier) và cột ứng với từng PM, như sau:

$$\Phi_i = \begin{pmatrix} v_{11}^i & v_{12}^i & \dots & v_{1m}^i \\ v_{21}^i & v_{22}^i & \dots & v_{2m}^i \\ \vdots & \vdots & \ddots & \vdots \\ v_{k1}^i & v_{k2}^i & \dots & v_{km}^i \end{pmatrix} \quad (10)$$

trong đó v_{ikm}^i là số lượng máy ảo được cấp phát cho ứng dụng i ở tầng k trên PM m .

Để bảo đảm hệ thống luôn có hiệu năng tốt, hệ thống nên tối đa hóa việc sử dụng tài nguyên của các PM một cách đồng đều. Nói cách khác, các máy ảo được phân bố đều trên các máy vật lý, khi đó hệ thống sẽ đạt trạng thái cân bằng. Để đo hiệu quả sử dụng tài nguyên của một PM, ta dùng công thức sau:

$$h_j = \frac{c_j^u}{c_j} + \frac{\eta_j^u}{\eta_j} + \frac{d_j^u}{d_j} \quad (11)$$

trong đó, c_j^u là số CPU đã sử dụng và c_j là dung lượng CPU của PM j , η_j^u là lượng RAM đã sử dụng và η_j là dung lượng RAM của PM j , d_j^u là số DISK đã sử dụng và d_j là dung lượng DISK của PM j .

Mức độ cân bằng tải của hệ thống được đo bằng công thức sau:

$$V = \frac{\sum_{j=1}^M (h_j - \bar{h})^2}{M} \quad (12)$$

trong đó $\bar{h}$ là hiệu năng trung bình.

Đối với các nhà cung cấp dịch vụ, để đạt được lợi nhuận cao, họ khai thác tối đa năng lực của PM, tránh lãng phí tài nguyên vật lý. Mức lãng phí của PM j trong hệ thống được tính như sau:

$$w_j = \frac{c_j^f}{c_j} + \frac{\eta_j^f}{\eta_j} + \frac{d_j^f}{d_j} \quad (13)$$

trong đó, c_j^f là số CPU còn trống (available) của PM j , η_j^f là lượng RAM còn trống của PM j , d_j^f là số DISK còn trống của PM j .

Tổng mức lãng phí của hệ thống được tính như sau:

$$W = \sum_{j=1}^M w_j \quad (14)$$

Chúng tôi xem việc điều chỉnh tài nguyên như một trò chơi và các ứng dụng như những người chơi. Giả sử mỗi người chơi có thể thay đổi chiến lược phân phối/di chuyển như một hành động điều chỉnh để di chuyển các máy ảo từ PM lỗi sang PM an toàn như sau:

$$a_i = \begin{cases} v_{km}^i - 1 \\ v_{ke}^i + 1 \end{cases} \quad (15)$$

Để biểu diễn sự đánh đổi (trade-off) giữa cân bằng tải và lãng phí tài nguyên, hàm lợi ích (pay-off function) của người chơi i khi phục vụ các máy ảo được biểu diễn như sau:

$$F^i(\Phi^i) = \frac{1}{\tau V_i + (1 - \tau) W_i}, \quad \forall \tau \in [0, 1] \quad (16)$$

Trong trò chơi này, hàm lợi ích của trò chơi có ảnh hưởng quan trọng đến quyết định chiến lược của người chơi và đến kết cục của trò chơi. Mỗi người chơi sẽ chọn một chiến lược nhằm tối đa hóa hàm lợi ích của mình, do đó hàm mục tiêu là:

$$\begin{aligned} & \text{Max} \sum_{i=1}^n F^i(\Phi) \\ \text{ràng buộc: } & \sum_{i=1}^n \sum_{x=1}^m \sum_{y=1}^k (v_{yx}^i) \leq c_x \end{aligned} \quad (17)$$

Cân bằng Nash (Nash equilibrium) của trò chơi là một chiến lược mà tại đó không người chơi nào có thể tăng thêm lợi ích khi các người chơi khác giữ nguyên chiến lược của họ. Khi đó, nếu chiến lược của người chơi i là chiến lược tối ưu, được ký hiệu là Φ_i , và chiến lược tối ưu của các người chơi khác được ký hiệu là Φ_{-i} , thì điểm cân bằng chiến lược Nash sẽ tuân theo điều kiện [29] như sau:

$$F_i(\Phi_{-i}^*, \Phi_i^*) \geq F_i(\Phi_{-i}^*, \Phi_i) \quad (18)$$

Trong một hệ đa tác nhân (multi-agent system), điểm cân bằng có thể không ổn định [30]. Ngoài ra, việc tìm ra tính hiệu quả Pareto (Pareto-efficiency) của cân bằng Nash là rất khó. Để giải quyết vấn đề này, hầu hết các thuật toán đều dựa trên các thuật toán metaheuristic. Các phương án gán máy ảo cho các PM khả thi dựa trên thuật toán tối ưu. Từ tập các phương án khả thi thỏa mãn điều kiện cân bằng Nash, thuật toán sẽ chọn ra phương án tốt nhất. Điều kiện dừng của thuật toán theo [15]:

$$\sum_{i=1}^n (F_i^{itr} - F_i^{itr-1})^2 < \varepsilon \quad (19)$$

4. THUẬT TOÁN DI CHUYỂN MÁY ẢO

4.1. Nguyên lý của Simulated Annealing

Simulated Annealing (SA) là một thuật toán heuristic dựa trên xác suất, được dùng cho một lớp bài toán nhằm tìm một số điều kiện tốt nhất trong một không gian tìm kiếm lớn. Thuật toán được tạo ra một cách độc lập bởi Scott Kirkpatrick, C. Daniel Gelatt, Mario P. Vecchi vào năm 1983, và bởi Vlado Černý vào năm 1985 [31]. Trong thuật toán SA, mỗi điểm s của không gian tìm kiếm tương tự như một trạng thái của một hệ vật lý nào đó, và hàm E biểu diễn nội dung bên trong (nội năng) của hệ ở trạng thái đó. Ở mỗi bước, luật heuristic sẽ xem xét và đưa ra một trạng thái liền kề s' của trạng thái hiện tại s , cùng một hàm xác suất quyết định giữa việc chuyển hệ sang trạng thái s' hay tiếp tục giữ trạng thái s . Xác suất này đưa hệ về trạng thái có mức năng lượng thấp hơn. Bước này được lặp lại cho đến khi hệ đạt được một trạng thái đủ tốt, hoặc cho đến khi không thể tiếp tục được nữa.

Áp dụng thuật toán luyện kim (metallurgical algorithm) để tìm một quyết định công bằng và hiệu quả cho trò chơi như sau:

- **Không gian trạng thái (State space)** là tập hợp tất cả các chiến lược (nói cách khác, tập các quyết định) của người chơi.
- **Hàm năng lượng (Energy function)** là hàm lợi ích của quyết định cho mỗi trạng thái, được tính theo công thức hàm mục tiêu.
- **Trạng thái kế tiếp (Next status)** được tìm bằng cách thay đổi ngẫu nhiên quyết định của người chơi. Ở lần lặp thứ hai của thuật toán, tài nguyên của người dùng có thể được di chuyển hoặc không trên nền IaaS. Thuật toán tìm lời giải lân cận (neighbor solution) cho biết cách tìm ra các trạng thái kế tiếp, vốn là một chiến lược cấp phát tài nguyên người dùng một cách ngẫu nhiên. Với mỗi yêu cầu tài nguyên máy ảo, một PM có tài nguyên còn trống để phục vụ yêu cầu này sẽ được chọn. Xác suất chọn ngẫu nhiên để cấp phát máy ảo trên một PM cụ thể tuân theo phân phối chuẩn (normal distribution).

Nhiệt độ (Temperature – T) sẽ được đặt là 10000 để bảo đảm độ chính xác của lời giải.

4.2. Thuật toán di chuyển máy ảo PFTSA

Algorithm 1: PFTSA

```

1  /* Input:
2   * T           : max temperature (nhiệt độ tối đa)
3   *             : control variable for stop condition algorithm
                   (biến điều khiển cho điều kiện dừng của thuật toán)
4   * Output:
5   */
6  // Inittial input alogrithm (Khởi tạo đầu vào của thuật toán)
7   $\Phi_{best} \leftarrow \text{CreateInitialSolution}()$ 
8   $\Phi_{current} \leftarrow \Phi_{best}$ 
9  WHILE ( StopCondition( ) ) DO
10     // Generate next solution (Sinh lời giải kế tiếp)
11      $\Phi_{new} \leftarrow \text{CreateNeighborSolution}( \Phi_{current} )$ 
12     // Calculate temperature (Tính nhiệt độ)
13     tempcurr  $\leftarrow \text{CalculateTemperature}( T, : )$ 

```

```

13      // Evaluate play-off function of the solution
      (Đánh giá hàm lợi ích của lời giải)
14      IF F(  $\Phi_{new}$  ) > F(  $\Phi_{current}$  ) THEN
15           $\Phi_{current} \leftarrow \Phi_{new}$ 
16          IF F(  $\Phi_{new}$  ) > F(  $\Phi_{best}$  ) THEN
17               $\Phi_{best} \leftarrow \Phi_{new}$ 
18          END
19      END
20      ELSE IF ( Exp( ( F( $\Phi_{current}$ ) - F( $\Phi_{new}$ ) ) / tempcurr ) > Rand() ) THEN
21           $\Phi_{current} \leftarrow \Phi_{new}$ 
22      END
23  END

```

(Ghi chú của người dịch: một số ký hiệu biến/lời giải trong mã giả gốc bị mờ; tên hàm và cấu trúc logic được giữ nguyên. Các tên biến lời giải như Φ_{best} , $\Phi_{current}$, Φ_{new} được điền theo đúng logic của thuật toán để dễ đọc, không làm thay đổi luồng xử lý.)

Thuật toán sinh lời giải khả thi mới như sau:

Algorithm 2: CreateNeighborSolution

```

1  /* Input:
2   * action: action for Application such as migration or not
      (hành động cho Ứng dụng, chẳng hạn di chuyển hoặc không)
3   * n: the number of applications (số lượng ứng dụng)
4   * m: the number of PMs (số lượng máy vật lý PM)
5   * constraints: the constraints of using resources IaaS
      (các ràng buộc về sử dụng tài nguyên IaaS)
6   * Output *
7  FOR (k = 1 to n) DO
8      IF (action == MIGRATE) THEN
9          FOR ( vm in pm_source ) THEN
10             // Create VM destination based on the config of VM source
              (Tạo VM đích dựa trên cấu hình của VM nguồn)
11             vm_des = createVM(vm_source)
12             // Find appropriate PM destination randomly
              (Tìm PM đích phù hợp một cách ngẫu nhiên)
13             pm_des = randomFindPM(1, m, constraints)
14             // Deploy VM destination on PM destination
              (Triển khai VM đích trên PM đích)
15             add(vm_des, pm_des, )
16             // Remove VM source out PM source
              (Gỡ VM nguồn khỏi PM nguồn)
17             remove(vm, pm_source, )
18          END
19      END
20  END

```

Hàm xác suất được định nghĩa bằng công thức:

$$P = \text{Exp} \left(\frac{F(\Phi^{current}) - F(\Phi^i)}{T} \right) \quad (20)$$

5. ĐÁNH GIÁ

Chúng tôi quan tâm đến bài toán di chuyển máy ảo, tức bài toán bảo đảm kỳ vọng của các bên liên quan (stakeholders) và được mô hình hóa bằng lý thuyết trò chơi. Thí nghiệm được thực hiện trên mô phỏng một trung tâm dữ liệu gồm 50 PM với cấu hình không đồng nhất, triển khai 30 ứng dụng ba tầng (three-tier) trong khoảng thời gian $t = 1000$. Các cấu hình khác nhau và được sinh ngẫu nhiên theo số cấu hình lớn nhất và nhỏ nhất (CPU, RAM, DISK) của các PM và VM, được trình bày trong Bảng 1. Tại mỗi thời điểm rời rạc t , các tham số đo lường của hệ thống bao gồm trạng thái của các PM và các ứng dụng. Để đánh giá hiệu quả của thuật toán di chuyển máy ảo, giả sử rằng tại mỗi thời điểm rời rạc t , thành phần dự đoán lỗi sẽ đánh giá trạng thái của các PM và tìm ra những máy ảo nào cần được di chuyển. Các yêu cầu di chuyển máy ảo sẽ được chuyển tới thành phần điều phối máy ảo (VM coordinator), vốn thực hiện việc di chuyển máy ảo. Các lời giải của thuật toán SA phụ thuộc vào các tham số ϵ và *cooling rate*, vốn sẽ được nghiên cứu trong mô phỏng này. Các tham số phù hợp của thuật toán PFTSA sẽ được đánh giá dựa trên mức độ cân bằng tải của hệ thống theo công thức (12) và mức độ lãng phí tài nguyên của hệ thống theo công thức (14). Cuối cùng, chúng tôi so sánh (benchmark) kết quả của thuật toán PFTSA với thuật toán PSOVM trong [32]. Môi trường thí nghiệm trên máy tính có cấu hình: 8GB RAM, Core i5.

Bảng 1. Cấu hình hệ thống dùng để sinh dữ liệu

	MaxCPU	MinCPU	MaxRam	MinRam	MaxDisk	MinDisk
Host	20	8	64	8	64	16
VM	5	1	6	1	6	1

Từ tri thức của các chuyên gia và nghiên cứu trước đó [33], chúng tôi chọn tập luật cho việc dự đoán lỗi PM dựa trên hệ thống mờ Takagi-Sugeno như sau:

Bảng 2. Tập luật dự đoán lỗi PM

Luật	Trạng thái: Tải (load)	Trạng thái: Nhiệt độ (temperature)	Dự đoán (1 là lỗi, 0 là không lỗi)
1	Thấp (Low)	Thấp (Low)	0
2	Thấp (Low)	Trung bình (Medium)	0
3	Thấp (Low)	Cao (High)	1
4	Trung bình (Medium)	Thấp (Low)	0
5	Trung bình (Medium)	Trung bình (Medium)	0
6	Trung bình (Medium)	Cao (High)	1
7	Cao (High)	Thấp (Low)	1
8	Cao (High)	Trung bình (Medium)	0
9	Cao (High)	Cao (High)	1

Hình 4. Tần suất lỗi của 50 PM tại thời điểm $t = 1000$

(Biểu đồ tần suất – histogram: trục hoành là số PM bị lỗi (Number of PM Fault), trục tung là tần suất (Frequency).)

Hình 4 mô tả tần suất lỗi của 50 PM tại thời điểm t . Theo đó, ta thấy trường hợp có 4 PM cùng lỗi tại một thời điểm là phổ biến nhất, với hơn 200 lần, còn số lần có tới 10 PM cùng lỗi là thấp nhất, với khoảng 15 lần.

5.1. Nghiên cứu tham số

Chúng tôi thay đổi ϵ từ 0.01 đến 0.09 với *cooling rate* = 0.5 để đánh giá thời gian thực thi của thuật toán PFTSA. Từ kết quả của Hình 5, giá trị ϵ ổn định tại 0.03. Tham số *cooling rate* sẽ được thay đổi từ 0.1 đến 0.9 để đánh giá hiệu quả của thuật toán PFTSA đối với mức độ cân bằng tải của hệ thống (V) và mức độ lãng phí tài nguyên của hệ thống (W), được thể hiện trong Hình 6 và Hình 7.

Hình 5. Thời gian thực thi của thuật toán PFTSA theo ϵ

(Biểu đồ hộp – boxplot: trục hoành là Epsilon (ϵ) từ 0.01 đến 0.09, trục tung là Thời gian thực thi tính bằng mili-giây (Execution time (ms)).)

Hình 6. Mức độ cân bằng tải của hệ thống L với cooling rate là 0.1 và 0.9

(Hai đồ thị theo thời gian t: đường INPUT so với đường COOLINGRATE=0.1 (bên trái) và COOLINGRATE=0.9 (bên phải), trục tung là V.)

Hình 7. Mức độ lãng phí tài nguyên của hệ thống W với cooling rate là 0.1 và 0.9

(Hai đồ thị theo thời gian t: đường INPUT so với đường COOLINGRATE=0.1 (bên trái) và COOLINGRATE=0.9 (bên phải), trục tung là W.)

5.2. So sánh với thuật toán PSO

Từ các kết quả ở trên, chúng tôi chọn $\epsilon = 0.03$ và cooling rate = [giá trị] cho thuật toán PFTSA, và chọn swarmsize = 35, $c_1 = c_2 = 2$, $W_UPPERBOUND = 1$, $W_LOWERBOUND =$ [giá trị] cho thuật toán PSOVM trong [32]. Trong kết quả so sánh, mức độ cân bằng tải của hệ thống theo PFTSA gần bằng với PSOVM, mức độ lãng phí tài nguyên cũng vậy, như thể hiện ở Hình 8 và Hình 9. Thời gian thực thi của thuật toán PFTSA nhỏ hơn thuật toán PSOVM, như ở Hình 10. Tuy nhiên, giá trị hàm mục tiêu của các chiến lược di chuyển tài nguyên tốt nhất theo thuật toán PSOVM lại lớn hơn thuật toán PFTSA, như ở Hình 11.

Hiệu quả của các thuật toán không chỉ phụ thuộc vào các tham số điều khiển mà còn phụ thuộc vào cách thiết kế bài toán di chuyển máy ảo. Hiệu quả còn phụ thuộc vào đặc điểm của từng thuật toán, vì mỗi thuật toán xây dựng chiến lược di chuyển máy ảo cho trò chơi theo một cách riêng. Điểm mạnh của thuật toán PFTSA nằm ở việc ngẫu nhiên hóa cách xác định các chiến lược di chuyển, qua đó tăng khả năng khám phá và tìm kiếm lời giải tốt nhất (hoặc gần tối ưu). Trong khi đó, ở thuật toán PSOVM, việc xác định các chiến lược di chuyển dựa trên vị trí, vận tốc của các phần tử (particles) và giá trị tối ưu của bầy đàn (swarm optimal value) giúp định hướng các chiến lược tìm kiếm.

(Ghi chú của người dịch: một vài giá trị số của cooling rate và $W_LOWERBOUND$ trong đoạn này bị mờ trong bản gốc nên được đánh dấu [giá trị]; không suy đoán.)

Hình 8. Mức độ cân bằng tải của hệ thống tại t = 1000

(Đồ thị theo thời gian t so sánh ba đường: INPUT, PFTSA, PSOVM; trục tung là V.)

Hình 9. Mức độ lãng phí tài nguyên của hệ thống tại t = 1000

(Biểu đồ hộp – boxplot so sánh ba thuật toán: INPUT, PFTSA, PSOVM; trục tung là W.)

Hình 10. Thời gian thực thi của thuật toán SA và PSO

(Đồ thị theo thời gian t so sánh PFTSA và PSOVM; trục tung là Thời gian thực thi (ms).)

Hình 11. Giá trị hàm mục tiêu của các chiến lược tài nguyên tốt nhất theo thuật toán PFTSA và PSOVM

(Biểu đồ hộp – boxplot so sánh ba thuật toán: INPUT, PFTSA, PSOVM; trục tung là F.)

6. KẾT LUẬN

Trong bài báo này, chúng tôi trình bày một cách tiếp cận chịu lỗi chủ động cho điện toán đám mây theo mô hình cơ sở hạ tầng như một dịch vụ (IaaS) dựa trên việc di chuyển máy ảo. Theo đó, thành phần dự đoán lỗi dựa trên hệ thống mờ Takagi-Sugeno, còn thành phần di chuyển máy ảo dựa trên thuật toán SA. Chúng tôi đề xuất một cách tiếp cận chịu lỗi chủ động thỏa mãn các bên liên quan thông qua việc khai thác tài nguyên một cách hợp lý. Lý thuyết trò chơi được áp dụng để mô hình hóa bài toán trò chơi di chuyển máy ảo. Lời giải tối ưu (hoặc gần tối ưu) được xấp xỉ bằng Simulated Annealing dựa trên cân bằng Nash. Trong phần đánh giá, chúng tôi trình bày các tham số ảnh hưởng đến hiệu quả của thuật toán PFTSA thông qua việc điều chỉnh các tham số ϵ và cooling rate. Với kết quả nghiên cứu tham số của PFTSA, việc so sánh kết quả của thuật toán PFTSA với thuật toán PSOVM được thể hiện ở các khía cạnh: mức độ cân bằng tải, tài nguyên bị lãng phí, thời gian thực thi và giá trị hàm mục tiêu. Điểm mạnh của PFTSA nằm ở việc

ngẫu nhiên hóa cách xác định các chiến lược di chuyển, qua đó tăng khả năng khám phá không gian lời giải. Trong nghiên cứu tiếp theo, chúng tôi sẽ nghiên cứu các thuật toán metaheuristic khác cho bài toán này.

7. TÀI LIỆU THAM KHẢO

- [1] Parhami, B.: ‘Defect, Fault, Error, ..., or Failure’, IEEE Transaction on reliability 1997
- [2] Hasan, M., and Goraya, M.S.: ‘Fault tolerance in cloud computing environment: A systematic survey’, Computers in Industry, 2018, 99, pp. 156-172
- [3] Jhawar, R., and Piuri, V.: ‘Fault tolerance and resilience in cloud computing environments’: ‘Computer and Information Security Handbook (Third Edition)’ (Elsevier, 2017), pp. 165-181
- [4] Lu, K., Yahyapour, R., Wieder, P., Yaqub, E., Abdullah, M., Schloer, B., and Kotsokalis, C.J.F.G.C.S.: ‘Fault-tolerant service level agreement lifecycle management in clouds using actor system’, 2016, 54, pp. 247-259
- [5] Li, Y., and Lan, Z.: ‘FREM: a fast restart mechanism for general checkpoint/restart’, Computers, IEEE Transactions on, 2011, 60, (5), pp. 639-652
- [6] Luo, Y., and Manivannan, D.: ‘Theoretical and experimental evaluation of communication-induced checkpointing protocols in and families’, Performance Evaluation, 2011, 68, (5), pp. 429-445
- [7] Marzouk, S., and Jmaiel, M.: ‘A survey on software checkpointing and mobility techniques in distributed systems’, Concurrency and Computation: Practice and Experience, 2011, 23, (11), pp. 1196-1212
- [8] Yuan, D., Yang, Y., Liu, X., and Chen, J.: ‘A data placement strategy in scientific cloud workflows’, Future Generation Computer Systems, 2010, 26, (8), pp. 1200-1214
- [9] Ghemawat, S., Gobioff, H., and Leung, S.-T.: ‘The Google file system’, in Editor (Ed.)^(Eds.): ‘Book The Google file system’ (ACM, 2003, edn.), pp. 29-43
- [10] Liu, H., Jin, H., Liao, X., Yu, C., and Xu, C.-Z.: ‘Live virtual machine migration via asynchronous replication and state synchronization’, parallel and distributed Systems, IEEE Transactions on, 2011, 22, (12), pp. 1986-1999
- [11] Vallee, G., Charoenpornwattana, K., Engelmann, C., Tikotekar, A., and Scott, S.L.: ‘A Framework for Proactive Fault Tolerance’, Proceedings of the Third International Conference on Availability, Reliability and Security (ARES 2008 - The International Dependability Conference)
- [12] Plankensteiner, K., Prodan, R., and Fahringer, T.: ‘A New Fault Tolerance Heuristic for Scientific Workflows in Highly Distributed Environments based on Resubmission Impact’, Fifth IEEE International Conference on e-Science, 2009
- [13] Zhao, W., Melliar-Smith, P.M., and Moser, L.E.: ‘Fault Tolerance Middleware for Cloud Computing’, IEEE 3rd International Conference on Cloud Computing, 2010
- [14] Bala, A., and Chana, I.: ‘Fault Tolerance-Challenges, Techniques and Implementation in Cloud Computing’, International Journal of Computer Science Issues, 2012, 9, (1)
- [15] Armbrust, M., Fox, A., and Griffith, R.: ‘A view of cloud computing’, Communications of the ACM, 2010, 53
- [16] Isermann, R.: ‘Fault-Diagnosis Applications’, Springer, 2011
- [17] Tchernykh, A., Schwiegelsohn, U., Alexandrov, V., and Talbi, E.-g.J.P.C.S.: ‘Towards understanding uncertainty in cloud computing resource provisioning’, 2015, 51, pp. 1772-1781
- [18] Wang, T., Zhang, W., Ye, C., Wei, J., Zhong, H., Huang, T.J.I.T.o.S., Man., and Systems, C.: ‘FD4C: Automatic fault diagnosis framework for Web applications in cloud computing’, 2016, 46, (1), pp. 61-75
- [19] Luo, Y., Wang, Z., Wei, G., Shen, B., He, X., Dong, H., and Hu, J.: ‘Fuzzy-logic-based control, filtering, and fault detection for networked systems: a survey’, Mathematical Problems in Engineering, 2015

- [20] Jhavar, R., and Piuri, V.: 'Fault tolerance and resilience in cloud computing environments': 'Computer and information security handbook' (Elsevier, 2017), pp. 165-181
- [21] Dean, D.J., Nguyen, H., Wang, P., and Gu, X.: 'PerfCompass: Toward Runtime Performance Anomaly Fault Localization for Infrastructure-as-a-Service Clouds', in Editor (Ed.)^(Eds.): 'Book PerfCompass: Toward Runtime Performance Anomaly Fault Localization for Infrastructure-as-a-Service Clouds' (2014, edn.), pp.
- [22] Schneider, C., Barker, A.D., and Dobson, S.A.: 'Evaluating unsupervised fault detection in self-healing systems using stochastic primitives', EAI Endorsed Transactions on Self-Adaptive Systems, 2015
- [23] Delpha, C., and Diallo, D.: 'Incipient fault detection and diagnosis: A hidden information detection Problem', in Editor (Ed.)^(Eds.): 'Book Incipient fault detection and diagnosis: A hidden information detection Problem' (IEEE, 2015, edn.), pp. 837-842
- [24] Dong, J., Zuo, D., Liu, H., and Yang, X.J.J.o.E.: 'DPHM: A fault Detection Protocol based on Heartbeat of multiple Master-nodes', 2007, 24, (4), pp. 544-549
- [25] Salvador, R., Otero, A., Mora, J., de la Torre, E., Sekanina, L., and Riesgo, T.: 'Fault tolerance analysis and self-healing strategy of autonomous, evolvable hardware systems', in Editor (Ed.)^(Eds.): 'Book Fault tolerance analysis and self-healing strategy of autonomous, evolvable hardware systems' (IEEE, 2011, edn.), pp. 164-169
- [26] Ghosh, D., Sharman, R., Rao, H.R., and Upadhyaya, S.J.D.s.s.: 'Self-healing systems—survey and synthesis', 2007, 42, (4), pp. 2164-2185
- [27] Engelmann, C., Vallee, G.R., Naughton, T., and Scott, S.L.: 'Proactive fault tolerance using preemptive migration', in Editor (Ed.)^(Eds.): 'Book Proactive fault tolerance using preemptive migration' (IEEE, 2009, edn.), pp. 252-257
- [28] Liu, J., Wang, S., Zhou, A., Kumar, S., Yang, F., and Buyya, R.: 'Using proactive fault-tolerance approach to enhance cloud service reliability', IEEE Transactions on Cloud Computing, 2016
- [29] Osborne, M.J., and Rubinstein, A.: 'A course in game theory' (MIT press, 1994. 1994)
- [30] Pendharkar, P.C.: 'Game theoretical applications for multi-agent systems', Expert Systems with Applications, 2012, 39, (1), pp. 273-279
- [31] Aarts, E., Korst, J., and Michiels, W.: 'Simulated annealing': 'Search methodologies' (Springer, 2014), pp. 265-285
- [32] Thanh, K.B., Xuan, L.M.H., Khac, C.N., Dac, H.H., Tran, V.P., and Cong, H.T.: 'An auto-scaling VM game approach for multi-tier application with Particle swarm optimization algorithm in Cloud computing', in Editor (Ed.)^(Eds.): 'Book An auto-scaling VM game approach for multi-tier application with Particle swarm optimization algorithm in Cloud computing' (IEEE, 2018, edn.), pp. 326-331
- [33] Hanh, T., Mao, B.D., and Khiet, B.T.: 'Research on Building Fault Detection Solution for IaaS Cloud Computing Based on Fuzzy Logic Algorithm', in Editor (Ed.)^(Eds.): 'Book Research on Building Fault Detection Solution for IaaS Cloud Computing Based on Fuzzy Logic Algorithm' (ASME Press, 2013, edn.), pp.